

Gemma 4 E4B QAT — Complete Optimization & Testing Guide

Jetson Orin Nano 8GB | Updated September 26, 2026 | Author: Walker Kirkpatrick

Covers: infrastructure optimization (21-config sweep), sampling parameters (51-test sweep), solar games comparison (13/19 games), 48K context testing, and single-shot HTML game prompt design lessons.

1. Hardware & Environment

Platform: NVIDIA Jetson Orin Nano 8GB (aarch64, Linux 5.15.199-tegra)

RAM: 8GB unified LPDDR5 (CPU + GPU shared). 7.4GB usable, 5.8GB with GUI off.

Power: Solar-charged Jackery Explorer 500 (100W panel, net 11.30W). 19.6W under load.

Model: Gemma 4 E4B QAT — gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf (4.0GB, Unsloth QAT UD-Q4_K_XL)

Architecture: Dense, 42 layers, 2 KV heads, 256 head_dim, 18 shared KV layers (24 effective), sliding_window 512, 262K vocab. Multimodal (text+image+audio).

Prerequisite: GUI must be OFF before launching model: `sudo systemctl stop gdm.service` (frees ~1.8GB)

Restart GUI: `sudo systemctl start gdm.service`

2. Optimized Server Launch Command

This command reflects ALL verified optimizations from the 21-config infrastructure sweep and 51-test sampling sweep:

```
GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 llama-server \  
-m ~/models/gemma-4-E4B-it-qat-UD-Q4_K_XL.gguf \  
--alias gemma-4-e4b-qat --host 127.0.0.1 --port 8091 \  
-ngl 99 -c 32768 -ctk f16 -ctv f16 \  
-b 512 -ub 512 -fa on --jinja --fit off -np 1 -t 6 \  
--temp 0.4 --top-p 0.95 --top-k 40 --repeat-penalty 1.0
```

For 48K context: change -c 32768 to -c 49152 (verified: 17.1 tok/s, 0.28GB margin, zero speed loss)

Launch via setsid (not terminal background wrapper, which adds 200MB overhead and causes OOM):

```
setsid ~/llama.cpp/build/bin/llama-server [flags] > logs/server.log 2>&1 &
```

3. Infrastructure Findings (21-Config Sweep)

Project: ~/projects/e4b-optimization/ — 21 configurations tested, zero OOM, all completed.

Methodology: Single-test-per-process orchestrator. OOM-safe runner. llama-bench + CLI generation tests.

3.1 Infrastructure Parameters

KV cache type	f16 vs q4_0	f16	+4% gen speed (18.5 vs 17.7 tok/s) on Gemma 4 sliding-window arch
Batch size	64 vs 512	512	+7% prompt processing (554 vs ~60 t/s). No effect on generation.
Context size	16K, 32K, 48K, 64K	32K standard, 48K extended	No speed penalty up to 48K f16. 64K f16 over budget.
Flash attention	on vs off	on	6x prompt processing speed. Essential.
Load mode	auto, mlock, mmap+mlock	auto	mlock and mmap+mlock both hurt performance.
Threads	4, 6, 8	6	No meaningful difference. 6 is safe default.
GPU layers	99 (all)	99	All layers on GPU for maximum speed.

3.2 Context Headroom Analysis (GUI off, 5.8GB available)

Standard	32K f16	5.12 GB	0.68 GB	17.1
Extended	48K f16	5.52 GB	0.28 GB	17.1
Max f16	64K f16	5.93 GB	Over budget	N/A
Max q4_0	64K q4_0	5.51 GB	0.29 GB	16.4 (-4%)

48K f16 is the sweet spot: +50% context, zero speed loss, 0.28GB safety margin. Verified with 2 real game generations.

4. Sampling Findings (51-Test Sweep)

Project: ~/projects/e4b-optimum-settings/ — 51 tests across 3 HTML project types (snake game, coffee landing, data dashboard). Server restart every 6 tests. Resume logic for interrupted runs.

Validation: Custom HTML validator scores 0-100 based on structure, completeness, features, and syntax balance.

4.1 Optimum Sampling Parameters

temperature	0.2, 0.4, 0.6, 0.8	0.4	0.4-0.6 sweet spot. Barely affects accuracy.
top_k	20, 40, 60, 100	40	Minimal effect. 40 is safe default.
top_p	0.85, 0.90, 0.95	0.95	No measurable difference across range.
repeat_penalty	1.0, 1.05, 1.1, 1.15	1.0	KEY FINDING: +9% speed (17.5 vs 16.1 tok/s), no accuracy loss.
thinking	on, off	off (/no_think)	Saves ~10% tokens, no quality loss. Scored higher on Snake game.

4.2 Repeat Penalty — Detailed Analysis

repeat_penalty=1.0 (disabled) is the single biggest sampling win: +9% generation speed with zero accuracy loss across all 3 project types.

At rp=1.0: 17.5 tok/s, avg score 98/100

At rp=1.1: 16.0 tok/s, avg score 97/100

At rp=1.15: 15.8 tok/s, avg score 96/100

The speed gain comes from the repeat penalty computation being skipped entirely, reducing per-token overhead.

5. Solar Games Comparison (E4B vs Ornith)

Project: ~/projects/e4b-solar-games-comparison/ — 19 HTML game prompts from the Solar-Powered LLM Games Report.

Method: Single-shot API calls only (no Serena/MCP/multi-turn). Server restart between EACH game. max_tokens=20000. curl-based harness.

Result: 13 of 19 games completed. All 13 E4B games scored 100/100. Ornith completed 8/13 (62%).

5.1 Head-to-Head Results

1. Tower Tycoon v2	5,711	5.5m	17.4	100	Pass
2. Neon Invaders	8,866	8.6m	17.2	100	Pass
3. Stellar Assault v2	8,889	8.6m	17.3	100	Pass
4. Flippy the Chip	10,519	10.3m	17.1	100	Pass
5. DS9 Defense	8,522	8.2m	17.2	100	Pass
6. Grind City	7,890	7.6m	17.2	100	Pass
7. Jungle Swing	9,749	9.5m	17.1	100	Pass
8. Dr. Mario v2	8,724	8.4m	17.2	100	Pass
9. Checkers Duel	9,036	8.8m	17.2	100	FAIL
10. Chess Arena	9,174	8.9m	17.2	100	FAIL
11. Miss Pac-Maze v2	10,308	10.5m	16.4	100	FAIL
12. Harbor Captain	10,016	8.8m	17.2	100	FAIL
16. Slam Arena v2	7,920	7.7m	17.2	100	FAIL

5.2 Summary Statistics

Completion rate	13/13 (100%)	8/13 (62%)	E4B +38%
Avg generation speed	17.1 tok/s	9.3 tok/s	E4B 1.85x faster
Avg score	100/100	N/A (failed)	E4B perfect
Games Ornith failed	0	5	E4B completed all
Avg output size	8,946 tokens	N/A	E4B consistent

E4B completed 5 games Ornith could not finish: Checkers, Chess, Pac-Maze, Harbor Captain, Slam Arena. The 4B QAT model outperforms the 9B model on HTML game generation — faster, more reliable, and produces complete games.

6. 48K Context Tests — Large Prompt Games

Two games tested with 48K context (49152 tokens, f16 KV cache) and larger, more complex prompts:

6.1 No Man's Sky: Lone Traveller

Project: ~/projects/nomanssky-e4b-48k/

Prompt size	9,788 chars (~2,444 tokens)
Modules specified	8 (planet, mining, survival, creatures, starship, scanning, HUD, procgen)
Completion tokens	13,780
Generation speed	16.7 tok/s (48K f16, only 2% slower than 32K)
Elapsed	823s (13.7 min)
HTML output	46,053 chars, 1,136 lines, 49 functions
Validation	8/8 checks passed, score 100, fully complete
Feature coverage	15/16 features (missing: day/night cycle)
Finish reason	stop (natural end, not truncated)

Verdict: Complete, playable game. All 8 modules implemented. The largest E4B output at time of testing.

6.2 EV Nova: Stellar Drifter

Project: ~/projects/evnova-e4b-48k/

Prompt size	13,032 chars (~3,253 tokens)
Modules specified	10 (physics, combat, systems, trading, planets, outfits, ships, missions, HUD, NPCs)
Completion tokens	14,597 (largest E4B output ever)
Generation speed	16.5 tok/s
Elapsed	882s (14.7 min)
HTML output	53,253 chars, 1,343 lines, 34 functions
Validation	8/8 checks passed, score 100, fully complete
Feature coverage	18/20 features (keyword check false negatives)
Finish reason	stop (natural end, not truncated)

Code review found 15 bugs: 3 critical (Math.random without (), no map click handler, no hyperspace jump function), 5 high (no port UI, stubbed trading/upgrades/missions/shipyard), 5 medium, 2 low.

7. Single-Shot HTML Game Prompt Design Lessons

These findings are derived from 13 solar games + 2 large 48K games = 15 total single-shot HTML game generations.

7.1 CRITICAL: All UI Must Be Canvas-Drawn

The single most important prompt design rule: ALL UI must be drawn on canvas, never split between canvas and DOM/HTML overlays.

Evidence: No Man's Sky prompt said 'draw everything on canvas' — produced a complete playable game (15/16 features). EV Nova prompt allowed HTML overlays for port menus — E4B built the engine (physics, combat, NPCs, audio) solidly but stubbed every menu/transaction screen (trade, outfitter, shipyard, missions all became empty functions with 'omitted for brevity' comments).

Root cause: E4B can implement canvas rendering + game logic in one shot, but cannot implement both canvas rendering AND HTML DOM menu systems in the same output. The split architecture exceeds its single-shot capacity.

7.2 Module Count: 8 Deep > 10 Shallow

No Man's Sky	8	8/8	15/16 (94%)	46K chars
EV Nova	10	3/10	18/20 (90%)	53K chars
Solar games avg	N/A	13/13	100%	~29K chars avg

8 modules with deep specification produces a more complete game than 10 modules with shallow specification. E4B stops naturally at ~14-15K tokens (finish_reason: stop, not truncated) — it scaffolds architecture but considers the game 'done' before filling in later modules.

7.3 Prompt Sizing for 48K Context

Larger prompts at 48K context work well. E4B produces proportionally larger and more complex output:

~1,500 tokens (solar games)	~29K chars avg	~27 avg	100%
~2,444 tokens (NMS)	46K chars	49	94%
~3,253 tokens (EV Nova)	53K chars	34	90%

48K context has zero speed penalty vs 32K (both f16). The larger context allows larger prompts, which produce larger, more complex games — but module count must stay at 8 to ensure completeness.

7.4 E4B Natural Stop Point

E4B stops generating at ~14-15K tokens with finish_reason: 'stop' — this is NOT truncation. The model considers its work complete. In the EV Nova game, 5 functions are empty stubs with comments like 'Implementation omitted for brevity' — E4B scaffolded the architecture but decided it was done before implementing the transaction/UI layer.

Implication: Design prompts so that all 8 modules can be fully implemented within ~14K tokens. Avoid specifying features that require complex UI flows (trade menus, ship purchase screens) — keep everything canvas-rendered with simple key-triggered actions.

8. OOM Prevention (9 Rules)

1. GDM must be off: `sudo systemctl stop gdm.service` (frees ~1.8GB RAM)
2. Restart server between EACH game (not every 6) to clear KV cache accumulation
3. Use `setsid` for long runs, not terminal background wrapper (adds ~200MB, causes OOM)
4. Drop caches between tests: `sudo tee /proc/sys/vm/drop_caches <<< 3`
5. `-np 1` (single slot): concurrent requests queue. Never run multiple harnesses at once.
6. `GGML_CUDA_ENABLE_UNIFIED_MEMORY=1` required for >4GB models on Jetson
7. `--fit off` required by llama.cpp build 0b1bad1+ (without it, server aborts with -ngl 99)
8. Monitor RAM: `free -h` before each test. If under 1GB free, stop and restart server.
9. 48K f16 context uses 5.52GB total — only 0.28GB margin. Do not increase to 64K f16 (over budget).

9. Hardened Test Harness Architecture

Three approaches tested for running single-shot game generation on E4B. Reliability improved with each iteration.

9.1 Approach Comparison

Python urllib (run_game.py)	Low	Harness dies mid-generation (OOM/gateway cleanup). HTTP connection breaks. Server generates into void. Result lost.	Failed
curl subprocess (run_game_v2.py)	Medium	curl as detached subprocess writes to file independently. Harness can parse afterward. Better, but still needs Python process alive to parse.	Better
Bash + setsid + curl (run_remaining_games.sh)	High	Server via setsid, curl writes to file, separate Python parser. Each component survives independently. Most resilient.	Recommended

9.2 Recommended Harness Pattern

1. Launch server via setsid (fully detached from terminal):

```
setsid llama-server [flags] > logs/server.log 2>&1 &
```

2. Wait for server with curl health check (not /health — use /v1/models):

```
curl -s http://127.0.0.1:8091/v1/models | grep -q gemma
```

3. Delete stale response files before each curl:

```
rm -f logs/response_${NAME}.json
```

4. Send request via curl with long timeout:

```
curl -s --max-time 7200 -X POST [endpoint] -d @request.json -o response.json
```

5. Parse response file with separate Python script after curl completes:

```
python3 parse_response.py response.json >> results.jsonl
```

6. Kill server, drop caches, proceed to next game

9.3 Critical Harness Rules

- Delete stale response files before each curl (race condition: old file parsed as 0 tokens)
- Ensure only one curl at a time against single-slot server (-np 1)
- sync_caches needs sudo: subprocess.run(['sudo', 'tee', '/proc/sys/vm/drop_caches'], input=b'3\n')
- Resume logic: check completion_tokens > 0 AND no error key to count as completed
- Server restart between EACH game prevents KV cache accumulation crash

10. Code Quality Review Findings

Deep code review of 13 solar game HTML outputs + 2 large 48K game outputs.

10.1 Solar Games (13 outputs reviewed)

12 of 13 games are clean code: well-formed HTML, balanced braces/parens/brackets, zero external dependencies, zero TODO/FIXME markers, no duplicate functions. All have canvas rendering, Web Audio API sound, and game loops.

1 bug found in 13 games: Checkers Duel (game 9) had a duplicate executeMove function and win condition never checked. 92% clean code rate.

10.2 EV Nova (detailed review)

CRITICAL	Math.random without ()	L488	Asteroid shapes broken (NaN radius)
CRITICAL	No map click handler	N/A	Cannot hyperspace jump (single-system only)
CRITICAL	No hyperspace jump fn	N/A	currentSystemIndex never changes
HIGH	No port UI rendered	L1112	Trade/outfitter/shipyard never appear
HIGH	updateTrading is stub	L889	No buy/sell logic exists
HIGH	updateShipUpgrades empty	L908	Upgrades never modify stats
HIGH	No mission generation	N/A	Mission board always empty
HIGH	No shipyard UI/purchase	N/A	Cannot buy new ships
MEDIUM	saveGame() every frame	L1319	60 localStorage writes/sec
MEDIUM	No refuel transaction	N/A	R key toggles flag only
MEDIUM	Hailing is a stub	L1200	Notification only, no dialog
MEDIUM	Boarding loot missing	L438	Comment only, no code
MEDIUM	No landing proximity	L1172	Can land anywhere
LOW	No B key handler	N/A	Cannot initiate boarding
LOW	Enemy loot missing	L836	Comment only, no code

Root cause: E4B built a solid space combat engine (physics, combat, NPCs, audio) but ran out of output budget before implementing the menu/transaction layer. The game has empty updateTrading, updateShipUpgrades functions — E4B scaffolded the architecture but did not fill in the interactions.

11. All E4B Projects

e4b-optimization	Infrastructure variable sweep (21 configs)	f16 KV +4%, batch 512 +7%, mlock hurts
e4b-optimum-settings	Sampling parameter sweep (51 tests)	rp=1.0 +9% speed, thinking off saves 10% tokens
e4b-solar-games-comparison	13/19 games vs Ornith	E4B 100% completion, 1.85x faster, 5 games Ornith failed
nomanssky-e4b-48k	NMS game at 48K context	15/16 features, 46K chars, score 100
evnova-e4b-48k	EV Nova game at 48K context	53K chars, 18/20 features, 15 bugs (menu stubs)
e4b-serena-settings	Serena MCP bridge + agent config	5 games tested, 38/38 features on Astro Miner
cemetery-halloween-e4b	Halloween cemetery game	Top-view navigation game
astro-miner-e4b	Space mining game	38/38 features, zero manual fixes
crypt-shadowking-e4b	Dungeon crawler	24/24 features, E4B succeeded where Ornith failed
flippybit-e4b	Flappy bird style game	Computer chip platformer
nova-3-game	Space trader game	45/45 features, 3 manual bug fixes

12. Files Updated with Permanent Settings

All 6 E4B project start_agent.sh files updated with f16 KV + b512 + sampling flags:

- cemetery-halloween-e4b/start_agent.sh
- astro-miner-e4b/start_agent.sh
- crypt-shadowking-e4b/start_agent.sh
- e4b-serena-settings/start_agent.sh
- flippybit-e4b/start_agent.sh
- nova-3-game/start_agent.sh

Also updated: agent_config.yaml, agent_bridge.py, serena_e4b_bridge.py

Settings saved to: Hermes memory, fact_store (#91 infra, #111 sampling), skill:
e4b-optimum-settings